

Woxsen University

School of Technology



B.Tech - Computer Science and Engineering (Artificial Intelligence and Machine Learning)

PROJECT BASED LEARNING REPORT

Title of the Project

Design and Implementation of a Database Management System for Microfinance Loan and Repayment Management System

Course Code: 25TU03MJM0

Course Name: Database Management Systems

Semester: 3

Academic Year: 2026-2027

Submitted by:

Name: Bhargav Goud Surgani

Roll Number: 25WU0102324

Section: whales

Under the Guidance of:

Faculty Name: Dr. Kiran Mayee Adavala

Designation: Professor

Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)

DECLARATION

We hereby declare that the Project titled "Design and Implementation of a Database Management System for Microfinance Loan and Repayment Management System" is an original work carried out by us under the guidance of the faculty member specified in this report at Woxsen University.

We confirm that this work has been prepared for academic purposes and has not been submitted in part or full for the award of any other academic qualification. All sources and references used in the preparation of this report have been acknowledged appropriately.

The database design, relational schema, MySQL implementation, and Python-Flask based user interface described in this report were developed as part of the project work. The report documents the implemented system, its database relationships, operations, testing, and observed outcomes.

Place: Woxsen University, Hyderabad

Date: 07-10-2026

Bhargav Goud Suragani 25WU0102324

CERTIFICATE

This is to certify that the Project titled "Design and Implementation of a Database Management System for Microfinance Loan and Repayment Management System" is a bonafide work carried out by the student of B.Tech - Computer Science and Engineering (Artificial Intelligence and Machine Learning), under the supervision and guidance of the faculty member, during the Academic Year 2026-2027.

The project demonstrates the design and implementation of a relational database for managing branches, loan officers, clients, loans, repayments, and transaction logs, together with a web-based interface connected to MySQL.

Guide Signature: _____

Faculty Name: Dr. Kiran Mayee Adavala

Designation: Professor

School of Technology, Woxsen University

ABSTRACT

The Microfinance Loan and Repayment Management System is a relational Database Management System designed to organize and manage the core operational information of a microfinance institution. The project addresses the need to maintain structured information about branches, loan officers, clients, loans, repayments, and transaction logs while preserving relationships and data integrity.

The system was designed using an Entity-Relationship model and converted into relational tables. The implemented database contains six main entities: BRANCH, LOAN_OFFICER, CLIENT, LOAN, REPAYMENT, and TRANSACTION_LOG. Primary keys identify individual records, while foreign keys establish the relationships between related entities. The database is implemented in MySQL.

A Python Flask web interface was developed to provide a practical front end for the database. The interface allows users to view records and perform insertion and deletion operations for all six tables. Related records are selected through database-driven dropdowns; for example, a client can be associated with an existing loan officer and a loan with an existing client.

Testing confirmed that changes made through the interface are reflected in MySQL and that records can be viewed, inserted, and deleted through the application. The project therefore demonstrates the complete flow from a web-based user action to Flask application logic and finally to persistent MySQL database operations.

Keywords: DBMS, MySQL, Microfinance, Relational Database, ER Diagram, Flask, Primary Key, Foreign Key, CRUD

TABLE OF CONTENTS

DECLARATION

CERTIFICATE

ABSTRACT

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Background

1.2 Motivation and Relevance to Real-World Problems

1.3 Scope and Organization of the Report

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Definition

2.2 Objectives

2.3 Constraints and Assumptions

CHAPTER 3: LITERATURE REVIEW / BACKGROUND

3.1 Relational Database Concepts

3.2 Database Normalization

3.3 Primary and Foreign Keys

3.4 DBMS-Based Financial Record Management

3.5 Web-Based Database Applications

3.6 Comparative Summary

CHAPTER 4: METHODOLOGY & SYSTEM DESIGN

4.1 Overall Architecture

4.2 Entity-Relationship Design

4.3 Relational Schema

4.4 Database Relationships

4.5 Tools and Technologies

CHAPTER 5: IMPLEMENTATION

5.1 Database Implementation

5.2 Branch Module

5.3 Loan Officer Module

5.4 Client Module

5.5 Loan Module

5.6 Repayment Module

5.7 Transaction Log Module

5.8 Web Interface Implementation

CHAPTER 6: RESULTS & DISCUSSION

6.1 Dashboard and Database Connectivity

6.2 Insert Operation Testing

6.3 Delete Operation Testing

6.4 Relational Data Validation

6.5 Discussion and Interpretation

CHAPTER 7: CONCLUSION & FUTURE SCOPE

7.1 Summary

7.2 Learning Outcomes

7.3 Future Enhancements

REFERENCES

APPENDIX A: DATABASE TABLE SUMMARY

APPENDIX B: PROJECT DEMONSTRATION CHECKLIST

CHAPTER 1

INTRODUCTION

1.1 Background

Microfinance institutions manage a large amount of structured information involving clients, lending officers, loan accounts, repayment transactions, and operational branches. When these records are maintained using disconnected files or unstructured data, it becomes difficult to maintain consistency, retrieve information quickly, and trace the relationship between a loan and its repayment history.

A Database Management System provides a systematic approach for storing, organizing, retrieving, and modifying related data. In this project, a relational database model is used because the information naturally forms relationships: a branch has multiple loan officers, an officer manages multiple clients, a client can have multiple loans, a loan can have multiple repayments, and repayment activity can be recorded in a transaction log.

The project combines database design principles with an application interface. MySQL is used as the database engine, while Python Flask provides the application layer and HTML, CSS, and JavaScript provide the user interface.

1.2 Motivation and Relevance to Real-World Problems

The motivation for the project is to demonstrate how a relational DBMS can solve a practical information-management problem. Microfinance operations depend on accurate client and loan information, timely repayment recording, and reliable transaction histories. A structured system reduces duplicate information, supports controlled relationships, and makes routine database operations accessible through a simple interface.

The project also demonstrates the complete DBMS development cycle: identifying entities, designing an ER model, converting it to relations, defining keys and constraints, populating tables, connecting an application to the database, and testing data operations.

1.3 Scope and Organization of the Report

The scope includes six relational entities and a web interface for viewing, inserting, and deleting records. The current prototype focuses on database operations and relationship handling rather than a full production banking or accounting platform. Advanced features such as authentication, automated

repayment schedules, payment gateways, notifications, and role-based access are outside the implemented scope.

Chapters 2 and 3 describe the problem, objectives, constraints, and background. Chapter 4 explains the methodology and system design. Chapter 5 describes implementation. Chapter 6 presents testing and results. Chapter 7 concludes the report and proposes future enhancements.

CHAPTER 2

PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Definition

The central problem addressed by this project is the organized management of interconnected microfinance data. The system must maintain consistent information about branches, officers, clients, loans, repayments, and transaction records while allowing users to perform common database operations through an accessible interface.

A suitable solution must ensure that each record has a unique identifier, related records reference valid parent records, and database changes made through the interface are persisted in MySQL. The application should also make the relational structure understandable by presenting each entity as a separate section.

2.2 Objectives

#	Objective	Success Criterion
1	Design a relational data model	Identify entities, attributes, primary keys and foreign keys.
2	Develop an ER model	Represent the six entities and their relationships.
3	Implement the database	Create the six tables in MySQL with appropriate data types and keys.
4	Connect the application to MySQL	Flask successfully reads and modifies database records.
5	Provide record viewing	All six tables can be viewed from the web interface.
6	Provide insertion	New records can be inserted through the UI.
7	Provide deletion	Existing records can be deleted through the UI.
8	Validate relational operations	Foreign-key based selections and database changes work correctly.

2.3 Constraints and Assumptions

- The prototype uses MySQL as the relational database management system.
- The web application is designed for local demonstration using Python Flask.
- The current UI provides insertion, viewing, and deletion; update functionality is not part of the demonstrated prototype.
- The six entities shown in the ER design are treated as the core database tables.
- User authentication and production-level authorization are outside the present scope.
- The system assumes valid parent records exist before dependent records are created.

CHAPTER 3

LITERATURE REVIEW / BACKGROUND

3.1 Relational Database Concepts

Relational databases represent information using tables consisting of rows and columns. Relationships between tables are established using keys. This model is appropriate for microfinance information because clients, loans, repayments, and organizational units have well-defined relationships.

3.2 Database Normalization

Normalization is used to organize relational data so that unnecessary redundancy and update anomalies are reduced. The project design separates branch, officer, client, loan, repayment, and transaction information into related tables.

3.3 Primary and Foreign Keys

A primary key uniquely identifies a row within a table. Foreign keys reference primary keys in related tables. In this project, `branch_id`, `officer_id`, `client_id`, `loan_id`, `repayment_id`, and `log_id` identify records in their respective tables.

3.4 DBMS-Based Financial Record Management

Financial and lending applications depend on reliable data storage because an individual loan may be associated with many repayment events. A relational model allows these events to be stored separately while retaining a clear link to the corresponding loan and client.

3.5 Web-Based Database Applications

A web application can act as an interface between users and a database. In the implemented prototype, Flask receives form submissions, executes SQL statements through the MySQL connector, commits successful changes, and returns updated records to the interface.

3.6 Comparative Summary

Concept	Role in the Project	Relevance
Relational Model	Represents entities as related tables	Organized storage
Primary Key	Uniquely identifies records	Record integrity
Foreign Key	Connects related entities	Relationship integrity
Normalization	Separates related information	Reduces redundancy

Microfinance Loan & Repayment Management System

MySQL	Stores persistent data	Database implementation
Flask	Handles application/database interaction	Web application layer
HTML/CSS/JS	Provides user interface	Usability and presentation

CHAPTER 4

METHODOLOGY & SYSTEM DESIGN

4.1 Overall Architecture

The implemented system follows a three-layer architecture. The presentation layer is the browser-based user interface. The application layer is the Python Flask server, which processes requests and executes SQL operations. The data layer is the MySQL database containing the six relational tables.

Layer	Components	Responsibility
Presentation Layer	HTML, CSS, JavaScript	Forms, navigation, tables and user interaction
Application Layer	Python, Flask, MySQL Connector	Routing, validation, SQL execution and responses
Data Layer	MySQL	Persistent relational data storage

4.2 Entity-Relationship Design

The ER diagram contains six entities: BRANCH, LOAN OFFICER, CLIENT, LOAN, REPAYMENT, and TRANSACTION LOG. The major relationship chain is Branch → Loan Officer → Client → Loan → Repayment → Transaction Log.

The ER design represents one-to-many relationships from BRANCH to LOAN_OFFICER, from LOAN_OFFICER to CLIENT, from CLIENT to LOAN, from LOAN to REPAYMENT, and from REPAYMENT to TRANSACTION_LOG. Primary and foreign key attributes are identified in the design.

Microfinance Loan & Repayment Management System

Entity–Relationship (ER) Diagram

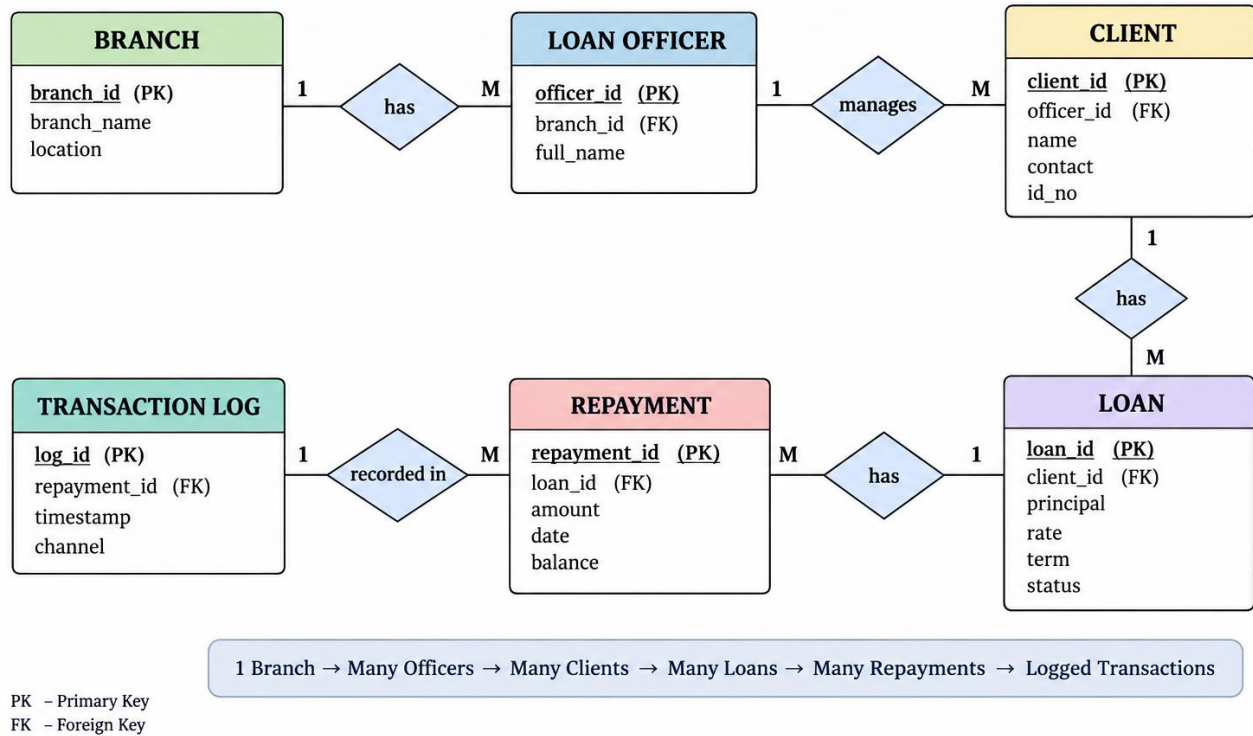


Figure 4.1: Final ER diagram of the Microfinance Loan and Repayment Management System

4.3 Relational Schema

Table	Primary Key	Important Foreign Key(s)	Main Attributes
BRANCH	branch_id	—	branch_name, location
LOAN_OFFICER	officer_id	branch_id	full_name
CLIENT	client_id	officer_id	name, contact, id_no
LOAN	loan_id	client_id	principal, rate, term, status
REPAYMENT	repayment_id	loan_id	amount, date, balance
TRANSACTION_LOG	log_id	repayment_id	timestamp, channel

4.4 Database Relationships

- One branch can have many loan officers.
- One loan officer can manage many clients.
- One client can have many loans.
- One loan can have many repayments.
- Repayment activity is associated with transaction log records.

4.5 Tools and Technologies

Technology	Purpose
MySQL 8.0	Relational database management
Python	Application programming
Flask	Web application framework
mysql-connector-python	Python-to-MySQL connectivity
HTML	Web page structure
CSS	User interface styling
JavaScript	Tab navigation and client-side interaction
Visual Studio Code	Development environment

CHAPTER 5

IMPLEMENTATION

5.1 Database Implementation

The database is implemented in MySQL under the database name `microfinance_db`. The six tables follow the relational structure defined by the ER design. Each table contains a primary key and relevant foreign-key references. Integer identifiers are used for main keys, VARCHAR fields for text values, DECIMAL fields for monetary values and rates, DATE for repayment dates, and DATETIME for transaction timestamps.

5.2 Branch Module

The Branch module manages `branch_id`, `branch_name`, and `location`. The interface displays existing branches and provides a form for inserting a new branch and an action for deleting a selected branch.

5.3 Loan Officer Module

The Loan Officer module manages `officer_id`, `branch_id`, and `full_name`. When adding an officer, the branch is selected from a database-driven dropdown.

5.4 Client Module

The Client module manages `client_id`, `officer_id`, `name`, `contact`, and `id_no`. The officer selection is populated from existing loan officers. The module supports viewing, inserting, and deleting client records.

5.5 Loan Module

The Loan module manages `loan_id`, `client_id`, `principal`, `rate`, `term`, and `status`. A client is selected from existing client records so the loan is associated with a known client.

5.6 Repayment Module

The Repayment module manages `repayment_id`, `loan_id`, `amount`, `date`, and `balance`. The loan selection is populated from existing loan records.

5.7 Transaction Log Module

The Transaction Log module manages `log_id`, `repayment_id`, `timestamp`, and `channel`. The repayment is selected from existing repayment records, while the channel can be selected from available methods.

5.8 Web Interface Implementation

The Flask application retrieves records from all six tables and passes them to the HTML template. Navigation tabs switch between Dashboard, Branches, Loan Officers, Clients, Loans, Repayments, and Transaction Logs. Each section contains a form for insertion and a table for viewing records, with a delete action for individual rows.

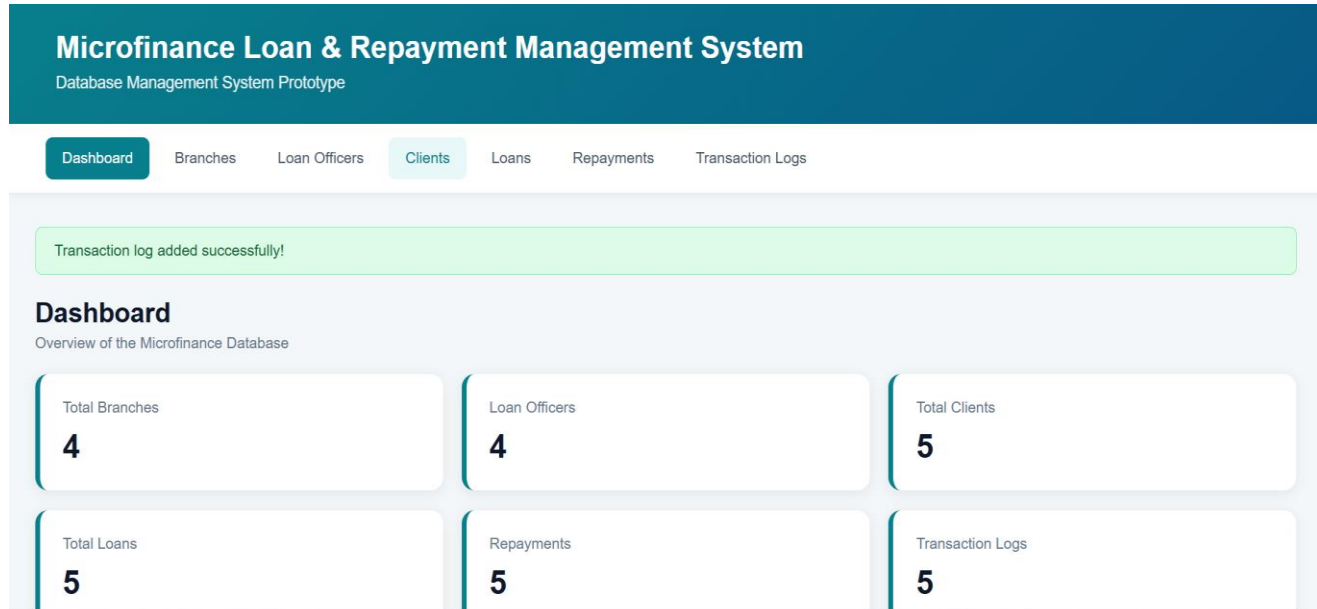


Figure 5.1: Web interface showing the multi-table DBMS dashboard .

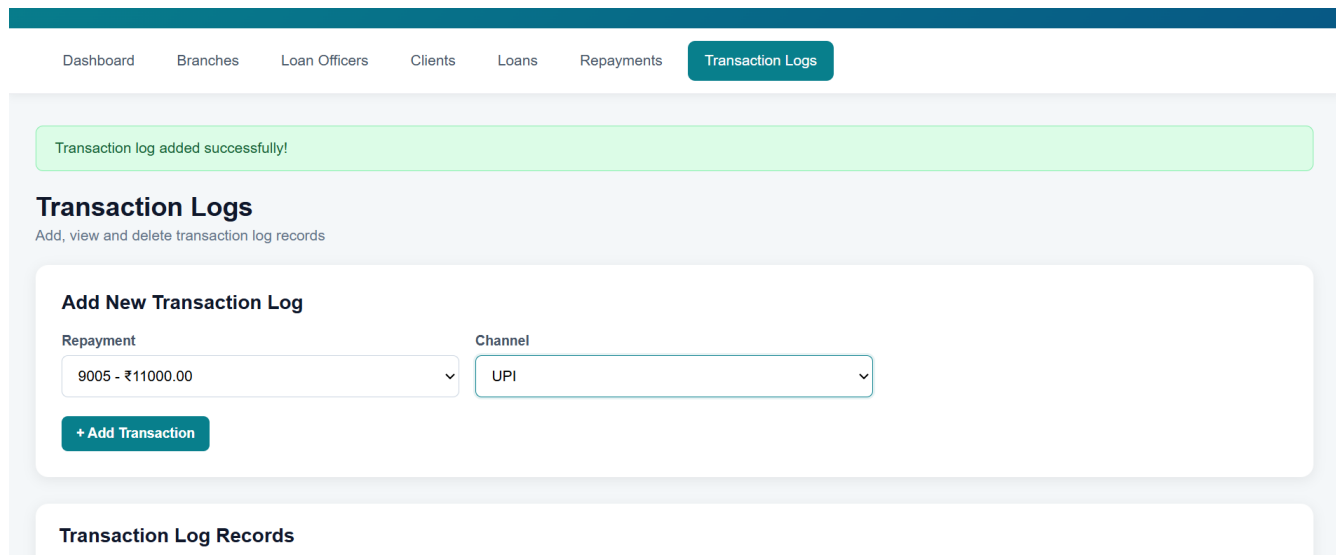


Figure 5.2: Related-record dropdowns and table operations.

CHAPTER 6

RESULTS & DISCUSSION

6.1 Dashboard and Database Connectivity

The completed prototype successfully connects the Flask application to the MySQL microfinance_db database. The dashboard provides a consolidated view of branches, loan officers, clients, loans, repayments, and transaction logs. The system status is displayed as connected when the database connection succeeds.

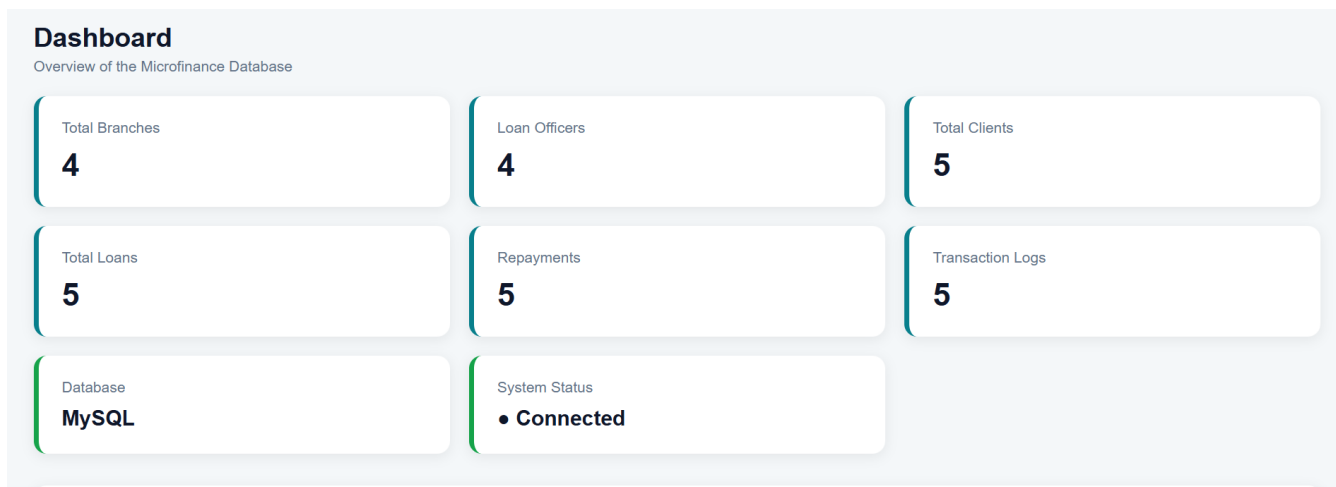


Figure 6.1: Dashboard showing database connectivity and entity counts.

6.2 Insert Operation Testing

Insertion was tested through the web interface. Flask receives the form submission, executes an INSERT statement, commits the transaction, and returns the updated table view.

Test Case	Operation	Expected Result	Observed Result
TC-01	Insert Branch	New branch appears in Branch table	Passed
TC-02	Insert Loan Officer	New officer appears with selected branch	Passed
TC-03	Insert Client	New client appears with selected officer	Passed
TC-04	Insert Loan	New loan appears with selected client	Passed
TC-05	Insert Repayment	New repayment appears	Passed

		with selected loan	
TC-06	Insert Transaction Log	New log appears with selected repayment	Passed

6.3 Delete Operation Testing

Deletion was tested through the interface. The delete action submits the selected primary key to Flask, which executes a DELETE statement and commits the change. The updated table is then displayed.

Test Case	Operation	Expected Result	Observed Result
TC-07	Delete Branch	Selected branch is removed	Passed
TC-08	Delete Loan Officer	Selected officer is removed when constraints permit	Passed
TC-09	Delete Client	Selected client is removed when constraints permit	Passed
TC-10	Delete Loan	Selected loan is removed when constraints permit	Passed
TC-11	Delete Repayment	Selected repayment is removed when constraints permit	Passed
TC-12	Delete Transaction Log	Selected log is removed	Passed

6.4 Relational Data Validation

The interface makes relationships visible during data entry. The Loan form displays existing clients, the Repayment form displays existing loans, and the Transaction Log form displays existing repayments. This demonstrates that the application operates on related relational data rather than isolated tables.

6.5 Discussion and Interpretation

The implemented system demonstrates the core DBMS workflow successfully: data is stored in relational tables, Flask retrieves the records, users interact through forms, SQL statements modify the database, and the updated state is reflected in the interface. The result is a functional prototype suitable for demonstrating database design, relationships, and basic data manipulation.

```
mysql> SELECT * FROM client;
```

client_id	officer_id	name	contact	id_no
1001	101	Suresh Kumar	9876543210	ID1001
1002	101	Lakshmi Devi	9876543211	ID1002
1003	102	Ravi Teja	9876543212	ID1003
1004	103	Anitha Rao	9876543213	ID1004
1005	104	Test User	9999999999	ID 1005

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM loan;
```

loan_id	client_id	principal	rate	term	status
5001	1001	50000.00	12.50	12	Active
5002	1002	75000.00	11.50	18	Active
5003	1003	40000.00	13.00	12	Completed
5004	1004	60000.00	12.00	12	Active
5005	1005	50000.00	11.00	12	Active

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM repayment;
```

repayment_id	loan_id	amount	date	balance
9001	5001	5000.00	2026-09-01	45000.00
9002	5001	5000.00	2026-09-15	40000.00
9003	5002	6000.00	2026-09-05	69000.00
9004	5003	4000.00	2026-08-20	0.00

Figure 6.2: MySQL command-line verification of records after UI operations.

CHAPTER 7

CONCLUSION & FUTURE SCOPE

7.1 Summary

The project successfully designed and implemented a relational Database Management System for a Microfinance Loan and Repayment Management System. The database model contains six connected entities: Branch, Loan Officer, Client, Loan, Repayment, and Transaction Log. The ER model was converted into a relational schema with primary and foreign keys.

The implementation uses MySQL for persistent data storage and a Python Flask application as the bridge between the database and a browser-based user interface. The completed UI provides separate sections for all six tables and supports viewing, insertion, and deletion of records. Testing confirmed that operations performed through the interface are reflected in the underlying MySQL database.

7.2 Learning Outcomes

- Understanding of entity identification and ER diagram design.
- Ability to convert an ER model into a relational schema.
- Practical use of primary keys and foreign keys.
- Understanding of normalization and relational data organization.
- Experience creating and querying MySQL tables.
- Experience connecting Python Flask applications to MySQL.
- Ability to implement web-based insertion, viewing, and deletion operations.
- Understanding of how application actions translate into database operations.

7.3 Future Enhancements

1. Add authentication and role-based access.
2. Add update/edit functionality for full CRUD support.
3. Add search, sorting, filtering, and pagination.
4. Implement automated loan schedules and installment calculations.
5. Automate outstanding balance calculation.
6. Add reports for active loans, completed loans, repayment performance, and branch summaries.
7. Add stronger validation and transaction handling.

8. Deploy the application securely using environment-based database credentials.
9. Add audit trails and improved transaction logging.
10. Add export functionality for reports in PDF or spreadsheet formats.

REFERENCES

- [1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, Pearson.
- [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, McGraw-Hill.
- [3] MySQL Documentation, MySQL 8.0 Reference Manual.
- [4] Flask Documentation, Flask Web Development Documentation.
- [5] Python Software Foundation, Python Documentation.
- [6] R. Ramakrishnan and J. Gehrke, Database Management Systems, McGraw-Hill.
- [7] Woxsen University, Project Based Learning / Database Management Systems course materials.

APPENDIX A: DATABASE TABLE SUMMARY

Table	Columns
BRANCH	branch_id, branch_name, location
LOAN_OFFICER	officer_id, branch_id, full_name
CLIENT	client_id, officer_id, name, contact, id_no
LOAN	loan_id, client_id, principal, rate, term, status
REPAYMENT	repayment_id, loan_id, amount, date, balance
TRANSACTION_LOG	log_id, repayment_id, timestamp, channel

APPENDIX B: PROJECT DEMONSTRATION CHECKLIST

Demonstration Item	Status
MySQL database created	Completed
Six relational tables implemented	Completed
ER diagram prepared	Completed
Flask application connected to MySQL	Completed
Dashboard displays database status	Completed
Insert operation tested	Completed
Delete operation tested	Completed
Records viewed through UI	Completed
Presentation III prepared	Completed